

FILES

When data is stored using variables, the data is lost when the program exits. “Files” discusses methods of working with files.

A File is a collection of information.

(or)

A sequential stream of bytes ending with an end-of-file marker is called a file.

Each file ends either with an end-of-file marker or at a specified byte number recorded in a system maintained, data structure. C supports two types of files called binary files and text files.

The difference between these two files is in terms of storage. In text files, everything is stored in terms of text i.e. even if we store an integer 54; it will be stored as a 3-byte string - “54\0”. For example, the integer value 1245 will be stored in 2 bytes for a binary file, while it will require 5 bytes in a text file.

To access any file, we need to declare a pointer to FILE structure and then associate it with the particular file. This pointer is referred as a file pointer and it is declared as follows:

```
FILE *fp;
```

Opening a file returns a pointer to a FILE structure (defined in <stdio.h>) that contains information, such as size, current file pointer position, type of file etc., to perform operations on the file.

The functions that have the ability to perform basic file operations are

- 1) Naming a file
- 2) Opening a file
- 3) Reading data from a file
- 4) Writing data to a file
- 5) Closing a file

To store data in a file we must specify certain things which include:

- 1) filename
- 2) Data structure
- 3) Purpose

Filename should be valid filename.

Data structure of a file is defined as FILE.

fopen:

allows to create a new file

syntax: fopen("filename","mode");

ex: fopen("rama.txt","r")

Filename is a valid name and mode can be one of the following.

r: opens a file for reading only.
w: opens a file for writing only
a: opens a file for appending data into it.
r+: allows both for reading and writing.
w+: allows both for writing and reading.
a+: allows both for appending and reading.

fclose(): When the processing of the file is finished, the file should be closed using the fclose() function, whose syntax is:

fclose(file pointer);

The return value is 0 if the file is closed successfully or a constant EOF, an end-of file marker, if an error occurred. This constant is also defined in <stdio.h>.

After opening the file, the next thing is we need to read or write the file. There are several functions defined in <stdio.h> header file for reading and writing the file. These functions can be categorized according to the type of data read or written on to a file.

File Input/Output functions are classified as:

- Character input/output functions
- String input/output functions
- Formatted input/output functions
- Block input/output functions.

Character input/output functions:

ANSI “C” provides a set of functions for reading and writing character by character or one byte at a time.

getc() is used to read a character from a file.
char_variable=getc(file_pointer);

putc() is used to write a character to a file.
putc(char_variable,file_pointer);

getc() and putc() are defined in <stdio.h> as macros.

Their syntax is as follows:

```
int putc(int ch, FILE *stream);  
int getc(FILE *stream);
```

The file pointer indicates the file to read from or write to.

The character ch is formally called an integer in putc() function but only the low order byte is used. On success putc() returns a character(in integer form) written or EOF on failure. Similarly getc() returns an integer but only the low order byte is used. It returns EOF when end-of-file is reached.

Apart from the above two macros, C also defines equivalent functions to read / write characters from / to a file. These are: fgetc() and fputc()

Their syntax is as follows:

```
int fgetc(FILE *stream);
```

`int fputc(int c, FILE *stream);`

feof(): To check the end of file.[feof() is a function]

Syntax: `int feof(FILE *fp);`

It returns 1 if end of file has been reached or 0 if not.

EOF: To check the end of file.[EOF is a macro]

String input/output functions:

If we want to read a whole line in the file, C provides some string input/output functions. We can read/write a set of characters at one time.

- `fgets( )` used to read a string from file.

Syntax: `fgets(string variable, length, FILE pointer);`

- `fputs( )` used to store a string into file.

Syntax: `fputs(string variable, FILE pointer);`

The integer parameter in `fgets( )` is used to indicate that at most num-1 characters are to be read, terminating at end-of-file or end-of-line. The end-of-line character will be placed in the string str before the string terminator, if it is read. If end-of-file is encountered as the first character, EOF is returned, otherwise str is returned. The `fputs( )` function returns a non-negative number or EOF if unsuccessful.

Formatted input/output functions:

If the file contains data in the form of digits, real numbers, characters and strings, and also if we want to write data in some specific format to a file, then we will use formatted input/output functions `fscanf()` and `fprintf()`.

These functions are used for formatted input and output. These are identical to `scanf()` and `printf()` except that the first argument is a file pointer that specifies the file to be read or written.

The syntax for these functions is:

`fscanf()`: is used to read formatted data from file
`fscanf(FILE *fp, "control strings", &arg-list);`

`fprintf()`: is used to store formatted data into file
`fprintf(FILE *fp, "format strings", arg-list);`

Block Input / Output functions:

read/write a block (specific number of bytes from/to a file). A block can be a record, a set of records.

`fread( )`

`fwrite( )`

These two functions allow reading and writing of blocks of data.

Their syntax is:

`fwrite()`: allows to write a block into a file

syntax:

```
size_t fwrite(const void *Ptr,size_t size,size_t n,file
*stream);
```

`const void *ptr:`

Specifies the address of a variable, which holds the actual data.

`Size_t size:`

Specifies the size of each record in bytes.

`sizeof:`

calculates the size of the expression.

sy: `sizeof()`;

`Size_t n:`

Specifies the number of records to be written to a file.

`file *file stream:`

Specifies the file pointer where the information is to be written.

`fwrite` function gets the address of a variable and writes `n` items of size into the specified file pointer.

fread(): reads a block of information from a file.

syntax: `size_t fread(const void *ptr,size_t size,size_t n, file *stream);`

fread function reads n items of size from the file into the structure specified by `const void *ptr`.

feof():

Determines whether the file pointer has reached the end of file or not. If it is end of the file feof() returns true otherwise false.

fseek(): fseek function is used to move the file pointer to a desired location within a file.

syntax: `fseek(file pointer,offset,position);`

File pointer is a pointer to the file concerned. Offset is a number or variable of type long and position is an integer number. The offset specifies the number of positions to be moved from the location specified by the position. The positions can take one of the 3 values.

SEEK_SET or 0	- beginning of file
SEEK_CUR or 1	-current position
SEEK_END or 2	- end of file

The offset may be positive it means that move forward, negative means move backward.

`ftell()`:

It takes a file pointer and returns a number of type long that corresponds to the current position. The function is useful in saving the current position of a file.

syntax: `ftell(file pointer);`

`rewind()`:

It takes a file pointer & resets the position to the beginning of the file.

Syntax: `rewind(file pointer);`